

+--pigpiod (pigpio daemon)

Step-by-step setup guide

Purpose

This guide installs **pigpio**, configures **pigpiod** as a systemd service, resolves common issues (PID file/locking, daemonization), and verifies Python connectivity.

Prerequisites

- Raspberry Pi or Debian-based system
 - `sudo` access
 - Network access (for package install and git)
-

1. Install packages and build from source (optional)

Use the system packages and also build from source to ensure latest binary in `/usr/local/bin`:

```
sudo bash -c 'apt update && apt install -y git build-essential pigpio-tools python3-pigpio
libpigpiod-if2-1t64 && \
cd /tmp && rm -rf pigpio && git clone https://github.com/joan2937/pigpio.git && cd pigpio &&
make && make install'
```

Notes:

- `make install` will place binaries/libraries under `/usr/local` and install Python bindings.

- If you prefer the distro package only, omit the `git clone && make` portion.
-

2. Create the systemd service file

Create `/etc/systemd/system/pigpiod.service` with the following contents. This version uses `Type=forking` which matches how `pigpiod` runs by default and allows `systemctl status` to show `active (running)`.

```
[Unit]
Description=Pigpio daemon
After=network.target

[Service]
Type=forking
ExecStart=/usr/local/bin/pigpiod
PIDFile=/var/run/pigpio.pid
Restart=on-failure
TimeoutStopSec=5
KillMode=process

[Install]
WantedBy=multi-user.target
```

Alternative (simple foreground)

If you prefer to run in foreground and let systemd track the process directly, use `Type=simple` and `ExecStart=/usr/local/bin/pigpiod -f` instead. Note that `Type=simple` may show `inactive` even while a child process keeps running unless `-f` is used and the binary behaves accordingly.

3. Reload systemd and enable the service

```
sudo systemctl daemon-reload
sudo systemctl enable --now pigpiod
sudo systemctl status --no-pager pigpiod
```

Expect either **active (running)** or a short-lived **inactive (dead)** depending on **Type** chosen and whether the binary forked. Proceed to next steps if the service isn't responding to clients.

4. Common issue: PID lock or stale instances

If logs show **Can't lock /var/run/pigpio.pid** or the service exits immediately, clean up stale processes and PID file:

```
sudo systemctl stop pigpiod
sudo killall pigpiod 2>/dev/null || true
sudo rm -f /var/run/pigpio.pid
sudo systemctl start pigpiod
sudo systemctl status --no-pager pigpiod
```

This ensures only one pigpiod instance runs and the PID file is fresh.

5. Verify Python connectivity

Run this short test to confirm the Python **pigpio** client can connect to the daemon:

```
python3 - <<'EOF'
import pigpio
pi = pigpio.pi()
print("Connected to pigpio:", pi.connected)
EOF
```

Expected output:

Connected to pigpio: True

If **False** or you see "Can't connect to pigpio at localhost(8888)", check the service status and logs with **journalctl -u pigpiod -e** and ensure the daemon is listening on the socket.

6. Optional: test GPIO pins (LED blink script)

A simple Python script to toggle a GPIO pin (replace `GPIO_PIN` with your pin number):

```
#!/usr/bin/env python3
import time
import pigpio

PIN = 17 # change to your GPIO

ti = pigpio.pi()
if not ti.connected:
    raise SystemExit('Cannot connect to pigpio')

try:
    ti.set_mode(PIN, pigpio.OUTPUT)
    for _ in range(10):
        ti.write(PIN, 1)
        time.sleep(0.5)
        ti.write(PIN, 0)
        time.sleep(0.5)
finally:
    ti.write(PIN, 0)
    ti.stop()
```

7. Troubleshooting tips

Check service logs:

```
journalctl -u pigpiod -e
```

-

Confirm binary path and version:

```
which pigpiod
/usr/local/bin/pigpiod -v
```

-

Confirm listener (port 8888) exists:

```
sudo ss -tlnp | grep 8888
```

-

If mixing distro package and `/usr/local` build, consider removing the distro package to avoid confusion:

```
sudo apt remove --purge pigpio pigpio-tools python3-pigpio libpigpiod-if2-1t64
sudo ldconfig
```

-

8. Commands cheat-sheet (copy/paste)

reload and enable service

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable --now pigpiod
```

stop/cleanup then start

```
sudo systemctl stop pigpiod
```

```
sudo killall pigpiod 2>/dev/null || true
```

```
sudo rm -f /var/run/pigpio.pid
```

```
sudo systemctl start pigpiod
```

status and logs

```
systemctl status --no-pager pigpiod
```

```
journalctl -u pigpiod -e
```

9. Notes & Recommendations

- Keep either the distro package or your locally-built `/usr/local` version to reduce confusion.
- Use `Type=forking` with `PIDFile=` for the most accurate `systemctl status` reporting if running the default pigpiod (which forks).
- If you want more advanced control (specifying port, logging), add command options to `ExecStart` (e.g. `/usr/local/bin/pigpiod -l` for local socket only, or `-g` to set a

specific port).

10. Script: `pwm_all_sync_busesafe.py`

Purpose

This Python script safely tests all available GPIO pins on a Raspberry Pi with a synchronized PWM “breathing” LED effect, skipping pins reserved for I²C, SPI, UART, HAT EEPROM, or any extra pins you define.

Requirements

```
sudo apt install pigpio python3-pigpio
sudo systemctl enable --now pigpiod
```

Configuration (inside the script)

Variable	Description	Default
<code>RESERVE_SPI</code> <code>1</code>	Reserve GPIO 16–21 for SPI1/I ² S	<code>False</code>
<code>EXTRA_RESER</code> <code>VED</code>	Extra pins used by your display/shield	<code>set()</code>
<code>FREQ_HZ</code>	PWM frequency for LED fading	<code>1000 Hz</code>
<code>INVERT</code>	Set True if LEDs are active-low	<code>False</code>
<code>CYCLE_SEC</code>	Duration of one full breathe cycle	<code>4.0 s</code>
<code>STEPS</code>	Number of PWM steps per cycle (smoothness)	<code>240</code>

Run

```
python3 pwm_all_sync_busesafe.py
```

Expected output example:

Reserved (not used): [0, 1, 2, 3, 4, 7, 8, 9, 10, 11, 14, 15]

Testing PWM on (BCM): [5, 6, 12, 13, 17, 19, 22, 23, 24, 25, 26, 27]
Press Ctrl+C to stop.

LEDs connected to those GPIOs will breathe in sync.

Safe exit behavior

- All tested GPIOs set to **input mode**
- Pull-ups disabled
- `pigpio` connection stopped cleanly

Troubleshooting

Symptom	Solution
Could not connect to <code>pigpio</code> daemon	<code>sudo systemctl restart pigpiod</code>
Some pins do nothing	Check wiring, BCM numbering, or reserved pins
PWM looks choppy	Increase <code>STEPS</code> or adjust <code>FREQ_HZ</code>